

Round table • semantic linting of PlantUML diagrams

26 min presenting, 18 min room discussion, 1 min buffer (0:44–0:45). If time runs short, take slides 8–10 at a minute each; keep the discussion on slides 7, 12, 15 and the vote on 16.

Part	Clock	#	Slide	Min	Of which discussion	Room
Opening 4 min	0:00–0:02	1	The round-table promise	2		
	0:02–0:04	2	Three questions for this round table	2	1	Warm-up: who writes, who builds on diagrams?
① The problem 15 min	0:04–0:06	3	What we mean by semantic correctness	2		
	0:06–0:09	4	Both render. Which branch approves?	3	1	Which branch approves?
	0:09–0:12	5	Both render. Which one is right?	3	2	Which one is right?
	0:12–0:14	6	Semantic quality: why it matters now	2		
	0:14–0:19	7	Today: reviewed by eye, found late	5	3	Q1: how do you find gaps today?
② The contribution 11 min	0:19–0:21	8	From linter to static model analysis	2		
	0:21–0:22	9	Code smells, model smells	1		
	0:22–0:24	10	Detect, measure, govern	2		
	0:24–0:25	11	Does the level matter? Measured: yes	1		
	0:25–0:30	12	The same pattern you already trust on code	5	3	Q2: what may a tool decide?
③ The value 10 min	0:30–0:30	13	Today, once more	0		
	0:30–0:33	14	Shift left: checked before it is built on	3		
	0:33–0:40	15	Diagram quality, built into the workflow	7	5	PoC: what must it show? Where's the gate?
Closing 4 min	0:40–0:44	16	A PoC on semantic linting: go or no-go?	4	3	Vote 👍 / 👎; if go: scope, criteria, owner, check-in
Buffer	0:44–0:45		Overrun, questions, close	1		

The round-table promise

Every diagram renders. Does it make sense?

The round-table promise

1 THE PROBLEM	PlantUML is being used in the organisation for creating different types of diagrams; the syntactic correctness of these diagrams can be validated using IDE plugins; however, maintaining semantic correctness is important but challenging, and very much uncharted terrain.
2 THE CONTRIBUTION	This roundtable invites engineers to discuss the value of introducing linting rules for PlantUML diagrams , helping ensure diagrams are not only syntactically correct, but also adhere to a defined semantic quality.
3 THE VALUE	Together, we will explore which conventions and quality checks deliver the most value, how diagram standards can be integrated into engineering workflows , and how well-structured diagrams can benefit both developers and AI-powered tools.
YOUR VIEW	Join the discussion to share your experiences, identify common pain points , and help define practical guidelines that improve collaboration, documentation quality, and automation across the software delivery lifecycle.

SAY

- Open with the invite as published: this is a discussion, not a product demo.
- The gap it names: PlantUML and IDE plugins already check syntax (does it render?); nobody checks semantics (does it make sense?).
- Each paragraph opens one part of the session: ① the problem, ② what a linter contributes, ③ the value in our workflow, and your view throughout.
- Two small fixes against the published text: "uncharted" and "are not only".

Next → Here is how we get there: three questions.

Three questions for this round table

Is semantic linting of diagrams worth a PoC?

Three questions for this round table

AGREE

1 The problem

Do we recognise it?

semantic correctness

common pain points

analysts & process engineers

developers & AI tools

→ a shared problem

UNDERSTAND

2 The contribution

What does a semantic linter add?

linting rules

conventions

quality checks

engineering workflows

→ a shared understanding

DECIDE

3 The value

Is it worth trying?

documentation quality

collaboration

automation

delivery lifecycle

→ a go / no-go for a PoC

PoC?
go / no-go

SAY

- Three questions, three outcomes: agree that there is a problem, understand what a semantic linter adds, decide whether a PoC is worth it.
- The chips are the keywords from the programme, sorted by the question they belong to.
- We end at a go / no-go for a PoC, not at adoption.

ASK

Warm-up, show of hands: who writes PlantUML diagrams? Who builds on someone else's diagram: a process, a design, code?

Next → First, what we mean by semantic correctness.

What we mean by semantic correctness

① The problem • beyond "does it render?"

What we mean by semantic correctness

Logical correctness 20% Is it logically sound? <i>A block that never closes; a state machine with no starting point.</i>	Completeness · gaps 30% Is everything that should be there actually there? <i>A process with no end; an error path nobody drew.</i>	Ambiguity · ambiguities 25% Could two readers take it two different ways? <i>Arrows with no label; conditions like "sometimes"; "TBD".</i>
Consistency 15% One convention, here and across diagrams? <i>One thing with two names, kinds or stereotypes.</i>	Traceability 5% Can you find it, cite it, and tell who owns it? <i>No title, no owner, no link to the requirement.</i>	Readability 5% Is it small enough to take in? <i>Too many lifelines or messages on one canvas.</i>

Underneath: syntax Does it render? Solved: PlantUML and its IDE plugins already check this.

SAY

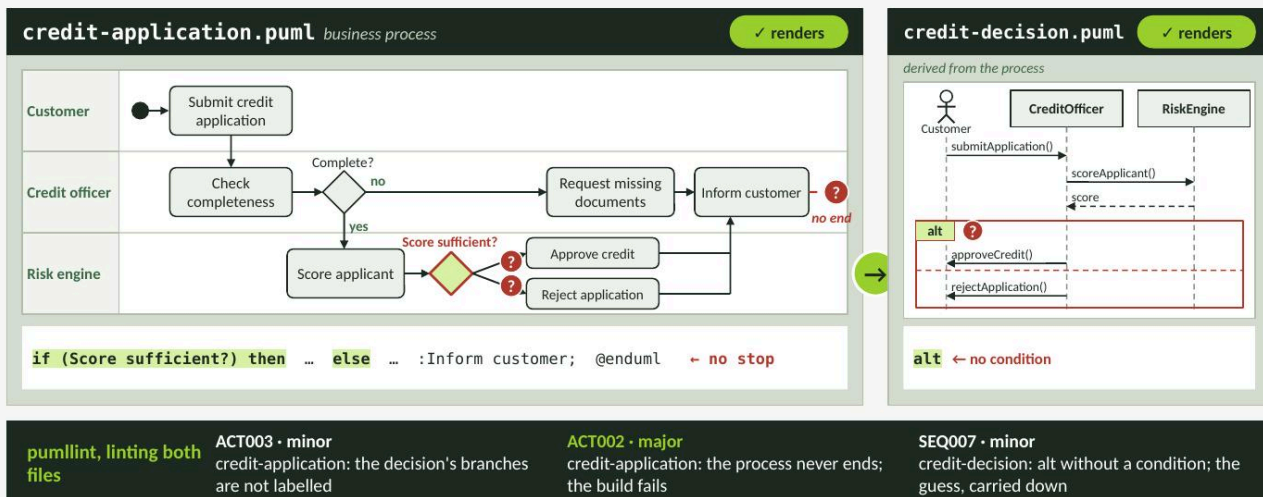
- Syntax sits underneath and is solved: PlantUML tells you whether the file renders.
- Semantic correctness is six questions a reader would otherwise answer with a guess. Walk them with one example each.
- "Gaps" are Completeness, "ambiguities" are Ambiguity: together 55% of the score, and they decide whether a diagram is ready for hand-off.
- The percentages are the weights pumllint uses to combine the six into one score.

Next → Let's see it in practice: a credit flow.

Both render. Which branch approves?

① The problem • a credit flow: one gap, passed on to the next diagram

Both render. Which branch approves?



SAY

- The first example: part of a credit flow as a process engineer would draw it, and the sequence diagram a designer derived from it. Both render.
- The process asks "Score sufficient?" but never says which branch is yes. The designer had to guess, and the guess shows up downstream as an alt block with no condition.
- pumllint reports both ends of the gap: ACT003 on the process (unlabelled branches) and SEQ007 on the sequence (alt without a condition). ACT002 adds that the process never reaches an end; it is major, so the build fails.
- pumllint does not know your credit policy. It flags that the diagram does not state it.

ASK

Before the reveal: which branch approves the credit? Then: who would have caught this today, and when?

TIP

The findings strip appears on the next click: ask first, then click.

Next → A second kind of problem only shows across diagrams.

Both render. Which one is right?

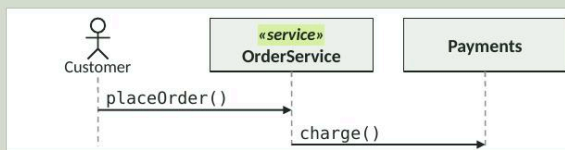
① The problem • each file is valid, together they contradict

Both render. Which one is right?

checkout.puml

✓ renders

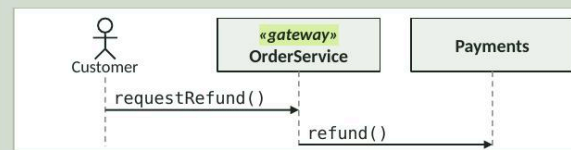
```
@startuml checkout
actor Customer
participant OrderService <<service>>
participant Payments
Customer -> OrderService : placeOrder()
OrderService -> Payments : charge()
@enduml
```



refund.puml

✓ renders

```
@startuml refund
actor Customer
participant OrderService <<gateway>>
participant Payments
Customer -> OrderService : requestRefund()
OrderService -> Payments : refund()
@enduml
```



pumlLint, linting both files together

XD002 • consistency • OrderService is «service» in checkout.puml and «gateway» in refund.puml: one entity, one stereotype

SAY

- The second example: two files, each valid, each renders. The syntax layer is satisfied, file by file.
- Reveal the strip only after the room has answered. Read across both files, pumlLint reports XD002, a Consistency finding: one entity, two stereotypes.
- It does not pick a side; it shows both places. A reviewer reading one file at a time never sees it.

ASK

Before the reveal: which one is right? Is OrderService a service with its own logic, or a gateway that passes calls through? Let two or three people answer.

TIP

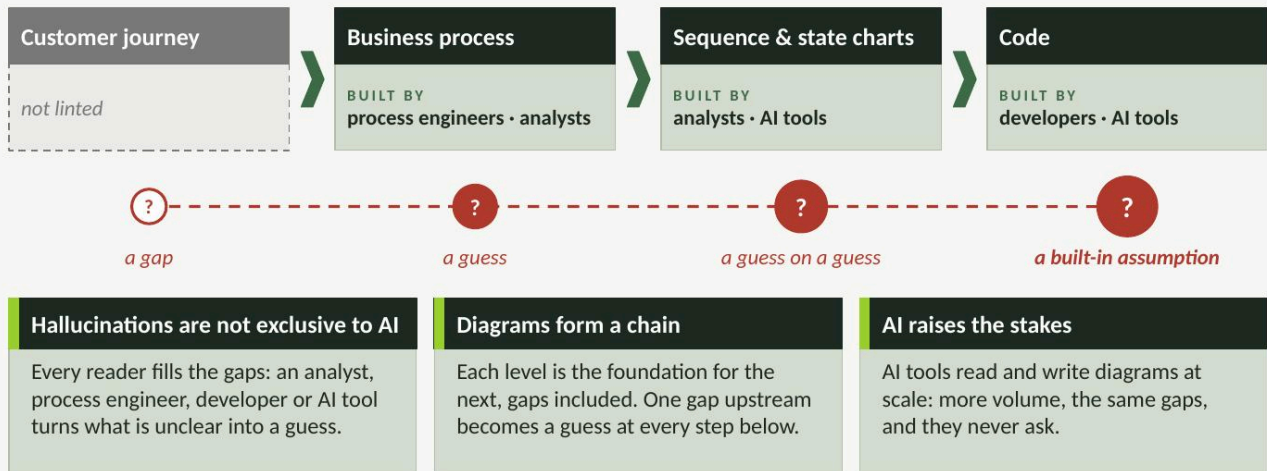
The XD002 strip appears on the next click: ask first, then click.

Next → Why this matters: someone builds on every diagram.

Semantic quality: why it matters now

① The problem • humans and AI tools build on what the diagram says, gaps and ambiguities included

Semantic quality: why it matters now



SAY

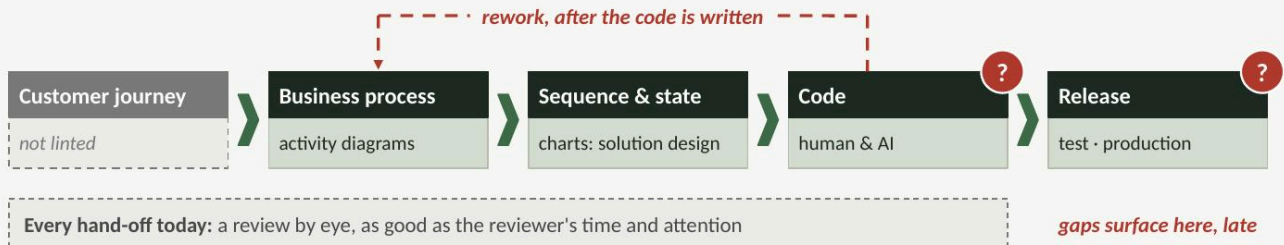
- The credit flow you just saw is one link of this chain. Walk it from left to right. The customer journey is not linted (there is no PlantUML journey type), so its gaps travel down unchecked.
- At every hand-off someone builds on the step before: process engineers and analysts, then analysts and AI tools, then developers and AI tools.
- Whatever is unclear becomes a guess; the guess becomes the foundation of the next step; in code it is an assumption that compiles and passes review.
- Filling gaps with guesses is not only an AI habit. AI raises the stakes: more volume, the same gaps, and it never asks.

Next → So how do we catch these gaps today?

Today: reviewed by eye, found late

① The problem • model quality is a subjective review concern

Today: reviewed by eye, found late



The later a gap is found, the more steps were built on it, and the further back the rework goes

Question: How do you find a diagram's gaps today, and at which step?

SAY

- Same value stream as later in the deck, without a quality station: every hand-off relies on a review by eye.
- Gaps that pass it surface in code, test or production. The fix travels back to the business process, and every step built on the gap is redone.
- That growth in rework is expected from how the chain works; we have not measured it.

ASK

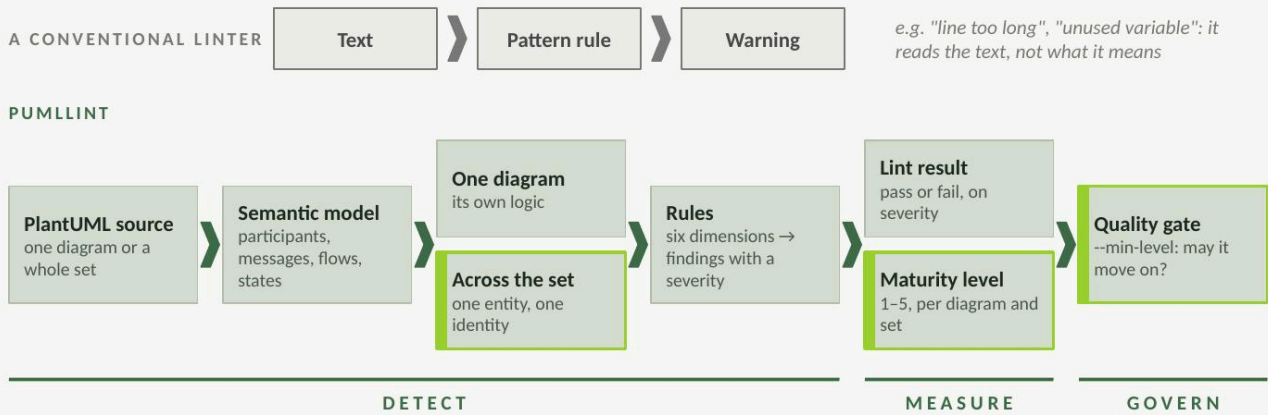
Question 1 on the slide: how do you find a diagram's gaps today, and at which step? Collect where, when and who.

Next → What could a tool add here?

From linter to static model analysis

② The contribution • a semantic static analyser for PlantUML models

From linter to static model analysis



What a conventional linter does not do. The rules stay the source of truth; the level adds them up, the gate applies your policy.

SAY

- A conventional linter matches patterns in text: line too long, unused variable.
- pumllint first turns each diagram into a model (who talks to whom, which flows, which states) and reasons over it, inside one diagram and across the whole set. Slide 5's finding only exists across files.
- Detect, findings with a severity, is what any linter does. pumllint goes two steps further: measure (a maturity level per diagram and per set) and govern (a gate the team sets).
- Hence "static model analysis" rather than just a linter.

Next → What do those findings look like?

Code smells, model smells

② The contribution · not always an error, always a reason to guess

Code smells, model smells

Code smell · SonarQube

Something in the code that makes it harder to understand, change or trust; not always a bug.
e.g. too complex · unreachable branch · unclosed resource

≈

Model smell · pumlLint

Something in the diagram that makes it harder to interpret, implement or trust: **a reason for the next reader to guess.**

Dimension	Diagram	Model smell	Rule	Severity
Logical correctness	sequence	A participant sends messages but was never declared	SEQ001	critical
Completeness	business process	The flow never reaches an end	ACT002	major
Ambiguity	state chart	A transition with no label: what triggers it?	STA003	minor
Consistency	across the set	One participant is a service here, a database there	XD001	major
Traceability	any	No link to the requirement or decision it implements *	GEN007	minor
Readability	any	Too many elements on one canvas (over 60)	GEN009	minor

* checked once your requirement-tag format is configured

SAY

- Borrow a term the room knows: a code smell is not always a bug, but it makes code harder to understand or trust.
- A model smell is the same for a diagram: not always an error, always a place where the next reader has to guess.
- One real rule per dimension, across diagram types. For process engineers, also ACT003: a decision whose branches are not labelled.
- Dark severity cells fail the build; the next slide shows what each severity does.

Next → What does each severity do?

Detect, measure, govern

② The contribution · every finding has a severity; every diagram gets a level

Detect, measure, govern

1 DETECT findings	Severity	Meaning	Effect
	Blocker	Unusable as a specification	Fails the build · max Level 2
	Critical	Broken structure	Fails the build · blocks Level 5
	Major	Breaks a mandatory standard	Fails the build · blocks Level 5
	Minor	Breaks a recommended convention	Reported · lowers the score
	Info	Advisory	Reported · lowers the score slightly
2 MEASURE maturity	Level	Name	Read it as
	5	Method-complete	Every dimension strong, no majors: preconditions for AI code generation met (codegen profile only)
	4	Precise	Complete and unambiguous: ready for hand-off
	3	Disciplined	No blockers: ready for review, not yet for hand-off
	2	Structured	Coherent enough to work on, although it may still contain blockers
	1	Sketchy	Don't build on it
3 GOVERN	The team sets the bar, e.g. Level 2 everywhere, Level 4 before hand-off: a diagram below it does not move on. Business processes: gate on findings, not on the level; no ambiguity rule covers activity diagrams yet.		

SAY

- Detect: blocker, critical and major fail the build; minor and info are reported and lower the score.
- Measure: the findings add up to a level per diagram. Level 4, Precise, is the hand-off bar: the next step can build on it without guessing.
- Level 5 needs the stricter code-generation profile. It means the preconditions for AI code generation are met, not that the code will be right.
- Govern: the team picks the bar and the gate enforces it. For business processes, gate on findings: no ambiguity rule covers activity diagrams yet, so their level can look better than it is.
- Proof from slide 4: the credit process scores Level 4 (Precise) while it carries a major finding that fails the build.

Next → Does the level actually matter?

Does the level matter? Measured: yes

② The contribution · the level predicts what happens downstream

Does the level matter? Measured: yes

Sequence diagrams at Level 1

~1 in 3

intended behaviours failed when the generated code ran

vs

Sequence diagrams at Level 2 and up

~1 in 10

intended behaviours failed when the generated code ran

Not an opinion: AI tools generated the code, and it was run against acceptance tests written beforehand.

Robust: held across three generators from two vendors and three prompt styles.

No prompt fixes it: better prompting helped untidy diagrams, never the Level 1 ones; it cannot add guards and failure paths the diagram never had.

Scope: sequence diagrams and AI-generated code only. **Expected, not measured:** less rework and a shorter lead time; a PoC is how we would find out.

Source: github.com/fdurieux/pumlLint · [EVIDENCE.md](#), "What the product may claim" · in plain English: [docs/evidence-explained.md](#), "Verdict 1"

SAY

- The one result we measured. AI tools generated code from sequence diagrams, and the code was run against acceptance tests written beforehand.
- From Level 1 diagrams about 1 in 3 intended behaviours failed; from Level 2 and up about 1 in 10. It held across three generators from two vendors and three prompt styles.
- Better prompting lifted moderately untidy diagrams, never the Level 1 ones: a prompt cannot add guards and failure paths the diagram never specified.
- Scope: sequence diagrams and AI-generated code only. Less rework and a shorter lead time are expected, not measured.

TIP

Source, if asked: EVIDENCE.md in the pumlLint repository, section "What the product may claim"; the plain-English version is docs/evidence-explained.md, "Verdict 1".

Next → If this sounds familiar, it should.

The same pattern you already trust on code

② The contribution · pumlLint and SonarQube, compared as linters

The same pattern you already trust on code

	pumlLint · on diagrams	SonarQube · on code	Where they meet
Analyses	diagram semantics, one diagram and across the set	code: control and data flow, security	-f sonar pumlLint → SonarQube ✓ Findings flow in: one dashboard, PR comments ✗ SonarQube does not analyse diagrams ► The maturity gate stays in pumlLint
Detects	model smells	bugs, vulnerabilities, code smells	
Rates	maturity level 1-5	ratings A-E	
Gates	- min-level	quality gate on new code	
Existing debt	ratchet: no diagram may get worse	Clean as You Code	
Runs in	editor (LSP) · command line · CI	IDE · CI scanner · server	

Question: Which of these would you trust a tool to decide, and which stays a human review?

SAY

- Compare them as linters, not by scope: both detect smells, rate what they find, gate on a threshold, and deal with existing debt without a big-bang cleanup.
- They meet in the pipeline: pumlLint exports its findings in SonarQube's import format (-f sonar), so they appear on the same dashboard and in pull-request comments.
- SonarQube does not analyse the diagrams itself. The maturity level arrives there as one informational issue per diagram, so the gate on the level stays in pumlLint.

ASK

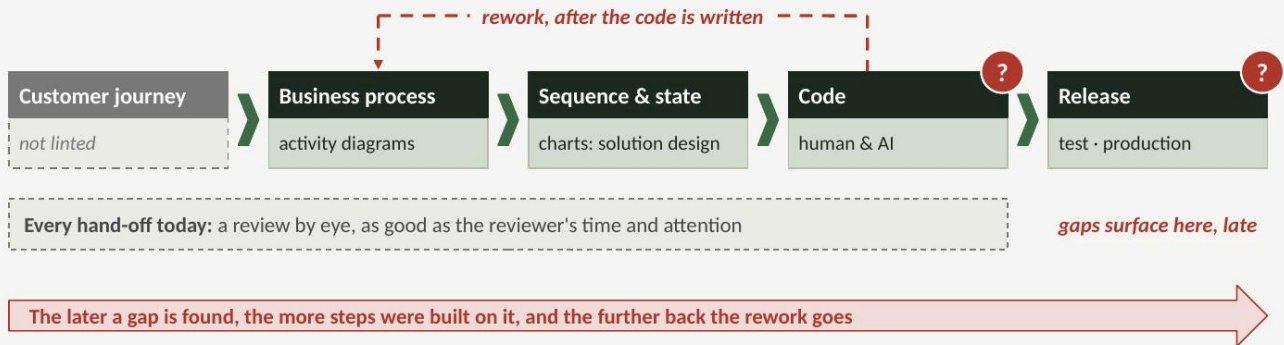
Question 2 on the slide: which of these would you trust a tool to decide, and which stays a human review?

Next → What would this change in our delivery?

Today, once more

③ The value • where we started

Today: reviewed by eye, found late



SAY

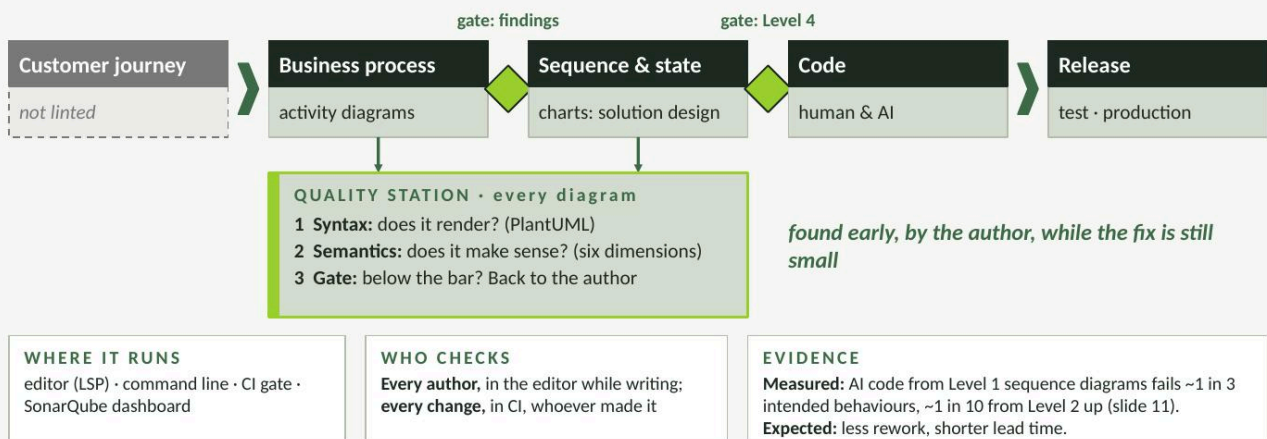
- Click straight through: this is slide 7 again, so the Morph to the next slide shows exactly what changes.

Next → What the quality station changes.

Shift left: checked before it is built on

③ The value • check each diagram before the next step builds on it

Shift left: checked before it is built on



SAY

- Same value stream as slide 7, same positions; two things changed.
- Each diagram passes a quality station before the next step builds on it: syntax, the six dimensions, then a gate. Business processes are gated on their findings, solution designs on Level 4.
- The rework loop is gone: the author finds the gap while the fix is still small. Every author checks in the editor; every change is checked in CI.
- Evidence: the measured result from slide 11 (AI code from sequence diagrams). Less rework and shorter lead time are expected, not measured: a PoC is how we would find out.

TIP

The Morph plays automatically from the recap slide (13).

Next → How would we start?

Diagram quality, built into the workflow

③ The value • from advice to gate, step by step, without a big-bang cleanup

Diagram quality, built into the workflow

On your own machine (with or without Git)

When	How
While writing	In any editor that supports LSP: one language server, no editor-specific plugins
On demand	From the command line: check a whole folder at once, with each diagram's findings and maturity level, plus an HTML maturity report

In the delivery pipeline (diagrams stored in Git)

When	How
On every change	A CI gate checks every changed diagram, whoever authored it; the severity of its findings decides whether the build passes
From existing tools	Business processes modelled in ARIS are converted to PlantUML and pass the same gate
Over time	Dashboards (SonarQube), a maturity report, and a badge per repository showing its weakest diagram's level

Step 1 • ADVISE
report only; nothing fails

Step 2 • RATCHET
record today's levels; fail only when a diagram gets worse

Step 3 • FLOOR
require a minimum level, e.g. Level 2 everywhere, Level 4 before hand-off

Questions

1. What would a PoC need to show to convince you?
2. Where should the gate sit, and which level for which diagrams?

SAY

- Two places, one tool. On your own machine: advice in the editor while you write, a full check from the command line with an HTML report.
- In the pipeline: a CI gate on every change, business processes converted from ARIS through the same gate, and dashboards, a report and a badge over time.
- Roll-out without a big-bang cleanup: advise (nothing fails), ratchet (nothing gets worse), floor (a minimum level).
- A PoC could start at step 1 on one team's diagrams.

ASK

The two closing questions: what would a PoC need to show to convince you? Where should the gate sit, and which level for which diagrams? Collect the answers: they become the success criteria of the PoC on the next slide.

Next → Time to decide.

A PoC on semantic linting: go or no-go?

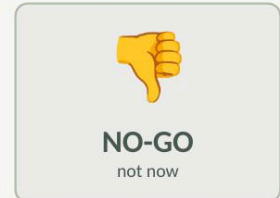
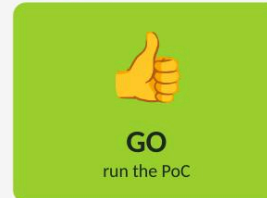
Closing · what we heard, and what we decide

A PoC on semantic linting: go or no-go?

WHAT WE DISCUSSED

1 The problem	Diagrams render, but not all of them make sense; every reader downstream fills the gaps.
2 The contribution	A semantic linter finds model smells, measures maturity and gates the hand-off.
3 The value	Checked in the editor and in CI; start with advice, no big-bang cleanup.

THE VOTE



No-go? Ask what would change your mind, and note it.

IF GO, WE AGREE BEFORE WE LEAVE

Scope	Success criteria	Owner & timebox	Next check-in
<i>which team, which diagrams</i>	<i>what must the PoC show?</i>	<i>who leads, how many weeks</i>	<i>when we look at the results</i>
...	...	...	...

SAY

- One line per part: diagrams render but not all make sense; a semantic linter finds model smells, measures maturity and gates the hand-off; it runs in the editor and in CI, starting with advice.
- Take the vote: thumbs up for go, thumbs down for no-go. Count both.
- If go, fill the four boxes before people leave: scope (one team, its diagrams), success criteria (from the answers on the previous slide), an owner with a timebox, and the date of the next check-in.
- If no-go, ask what would change people's minds and write it down: that is the input for a next attempt.

ASK

Thumbs up or thumbs down: do we run a PoC? Then, if go: who brings the diagrams, and what must the PoC show?

Appendix • A PoC on semantic linting: go or no-go?

Appendix • pumllint for the PoC • what we decide

A PoC on semantic linting: go or no-go?

WHY, IN THREE LINES

1 The problem	Diagrams render, but not all make sense; every reader downstream, human or AI, fills the gaps.
2 The contribution	A semantic linter finds model smells, gives each diagram a maturity level and gates the hand-off, in the editor and in CI.
3 Measured	AI code from Level 1 sequence diagrams fails ~1 in 3 intended behaviours; from Level 2 up, ~1 in 10.

YOUR TEAM

Team / squad	...
Team contact	<i>name, role</i>
Raised by	<i>round-table attendee</i>
Date	...

Your answer	☐ Go ☐ No-go ☐ Not yet What would change our mind?
--------------------	--

IF GO, WE AGREE ON

Scope	Success criteria	Owner & timebox	Next check-in
<i>which team, which diagrams</i>	<i>what must the PoC show?</i>	<i>who leads, how many weeks</i>	<i>when you look at the results</i>

Return to: [presenter name] · [email] · by [reply-by date]

Getting started: the three pumllint slides that follow

SAY

- Not presented: a one-page form for attendees to take back to their own squad or team lead, together with the three pumllint slides that follow.
- It carries the why in three lines, so a team lead who was not in the room can decide: team, contact, the answer (go, no-go, not yet) and, if go, the same four agreements as on the closing slide.
- Answers go back to [presenter name] by [reply-by date]; the address is in the lime strip.

TIP

Send slides 17 to 20 as one PDF (team-handout.pdf), or print them.

Appendix • Get started in ten minutes

Appendix • pumllint for the PoC • install and first run

Get started in ten minutes

SIX COMMANDS

```
$ pip install pumllint           # or pipx / uv tool install
$ pumllint diagrams/             # findings, with severity
$ pumllint score diagrams/       # maturity level per diagram
$ pumllint score diagrams/ -f html -o report.html
$ pumllint --list-rules          # everything it checks
$ pumllint lsp                   # findings in your editor
```

Python 3.11 or later • no runtime dependencies (PyYAML only for a YAML config). Read more in the repository: README • docs/findings-and-scores.md • docs/rule-map.md • docs/business-processes.md

WHAT YOU GET BACK

Findings
rule, severity and line, e.g. ACT002 • major

A maturity level
per diagram and per set, with what it takes to reach the next one

An HTML report
one self-contained page, to share or attach to a review

Exit codes for CI
0 clean • 1 findings at or above --fail-on (default major) • 2 usage error

SAY

- Not presented: hand-out for the PoC team. Six commands from the README quick start cover a first run.

Appendix • Fit it to your team

Appendix • pumllint for the PoC • configure, automate, connect

Fit it to your team

Configure

pumllint.yaml (or .toml / .json) is picked up automatically:

```
rules:
  unnamed-diagram: false
  participant-naming:
    severity: major
  requirement-link:
    pattern: "REQ-\d+"
```

One exception in one diagram: ' pumllint: disable=SEQ006

Automate

GitHub Actions, pinned to a release:

```
- uses: fdurieux/pumllint@v0.33.0
  with:
    command: score
    paths: docs/diagrams
    baseline: maturity.json
    min-level: "2"
```

Or per commit: the pre-commit hooks pumllint and pumllint-score.

Connect

SonarQube
-f sonar writes SonarQube's import format: findings on the same dashboard and in pull requests.

ARIS
the separate aris2pumll tool converts processes to PlantUML; they pass the same gate.

Editors
pumllint lsp in any editor that supports LSP; no editor-specific plugins.

Step 1 • ADVISE report only, no gate

Step 2 • RATCHET --baseline maturity.json

Step 3 • FLOOR --min-level 2

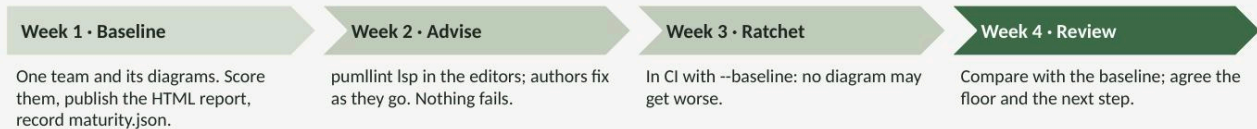
SAY

- Not presented: hand-out for the PoC team. One config file, a GitHub Action or pre-commit hooks, and the roll-out mapped to three commands.

Appendix • A four-week PoC, and what to measure

Appendix • puml lint for the PoC • a suggested plan

A four-week PoC, and what to measure



What to measure

- Findings per dimension, week 1 against week 4
- Sequence and state diagrams at Level 4; processes without a major finding
- Time to fix a finding, as the authors report it
- Noise: findings the team disagrees with, and the config changes they lead to
- Feedback from authors and reviewers
- Rework found downstream: the expected benefit, measured here for the first time

Success criteria

Agreed on the closing slide: what must the PoC show? Write them here before week 1, so week 4 compares against them.

A proposal to adapt: the length, the team and the measures are the PoC owner's call.

SAY

- Not presented: hand-out for the PoC team. A suggested plan; the measures give the expected benefits a first measurement.